

CLICK PLC Six-Floor Elevator Trainer

Directional Collective Control as the Central Teaching Concept

One-scan triggers, persistent requests, direction memory, 5-second inter-floor travel, physical/HMI door-close authorization, intermediate I/O images, and single-place physical output ownership.

Training scope - not a passenger elevator controller

This document describes a bench-top PLC demonstration and software simulation. It is **not** suitable for moving people. A passenger elevator requires certified safety hardware and code-compliant functions such as redundant door interlocks, brakes, overspeed protection, final limits, inspection and fire-service modes, emergency communications, load handling, and independently supervised safety circuits. The logic below intentionally focuses on scheduling, state memory, and PLC troubleshooting.

Three design rules used throughout

1. Trigger first. Physical and HMI commands are converted into one-scan events before they affect sequence memory. **2. Intermediate bits isolate the program from hardware.** Main logic never directly uses an X or writes a Y. **3. Physical outputs have one writer.** The six lamp commands are decoded once, and Y001--Y006 are written only on the I/O page. These rules make online debugging much easier because every signal has a clear owner and a clear path.

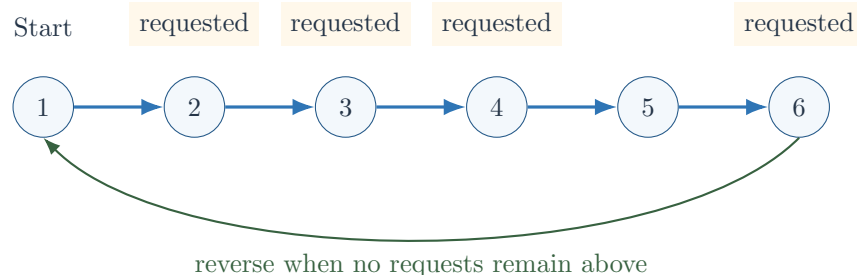
1 What Directional Collective Control Means

The trainer stores every requested floor in six persistent request bits. It does *not* pick one destination and ignore the rest of the queue. Instead, the car keeps a direction memory and collects requested floors in that direction in floor order. It reverses only when no pending request remains farther ahead.

Suppose the car starts at Floor 1 and requests for Floors 3, 6, 4, and 2 are present before the car passes them. A fixed first-in or low-floor-first target scheme can produce unnecessary reversals. The directional collective result is:

1 → 2 → 3 → 4 → 6

If Floor 2 is requested only after the car has already passed it while moving upward, that request remains stored and is served on the downward sweep.



Why this is a good six-floor teaching algorithm

The algorithm avoids needless direction changes, accepts new requests while the car is already moving, and demonstrates the same basic “continue the sweep, then reverse” idea used by selective-collective elevator systems. It does not mathematically minimize every passenger wait, but on a six-floor single-car trainer it is much more realistic than dispatching one fixed target at a time and is still transparent enough to troubleshoot rung by rung.

2 Operator and HMI Behavior

The physical buttons X001--X006 and the HMI floor command both create floor requests. A momentary physical pushbutton on X007, or the HMI command, supplies the deliberate *Door Close / Ready to Go* command. Either door-close rising edge is the departure authorization; a floor request by itself does not force the car to leave an open-door stop.

HMI object	CLICK item/value	Meaning
Floor 1--6 buttons	DS30 = 1--6	Momentary request. Write the chosen floor, then return DS30 to 0.
Door OPEN / Not Ready	DS31 = 0	Operator/HMI is not authorizing departure.
Door CLOSE / Ready	DS31 = 1	Rising edge creates one close/ready trigger and may start travel.
Physical Door CLOSE	X007	Momentary pushbutton; rising edge creates the same close/ready trigger.
Current Floor	DS1	Read-only cabin position, 1--6.
Direction	DS3	Read-only: 0 idle, 1 up, 2 down.
Pending requests	C101--C106	Read-only queue indicators.
Travel active	C120	Read-only motion state.
Door open / closed	C174 / C170	Read-only simulated door state.
Floor lamps	Y001--Y006	Read-only physical output image.

HMI command convention

Configure DS30 as a momentary command: selected floor on press and 0 on release. Likewise, after each stop, return DS31 to 0 before issuing the next 0-to-1 close/ready transition. Wire X007 as a normally-open momentary pushbutton so it returns off when released. Do not hold either close source on; the combined signal must return false before another rising edge can be recognized.

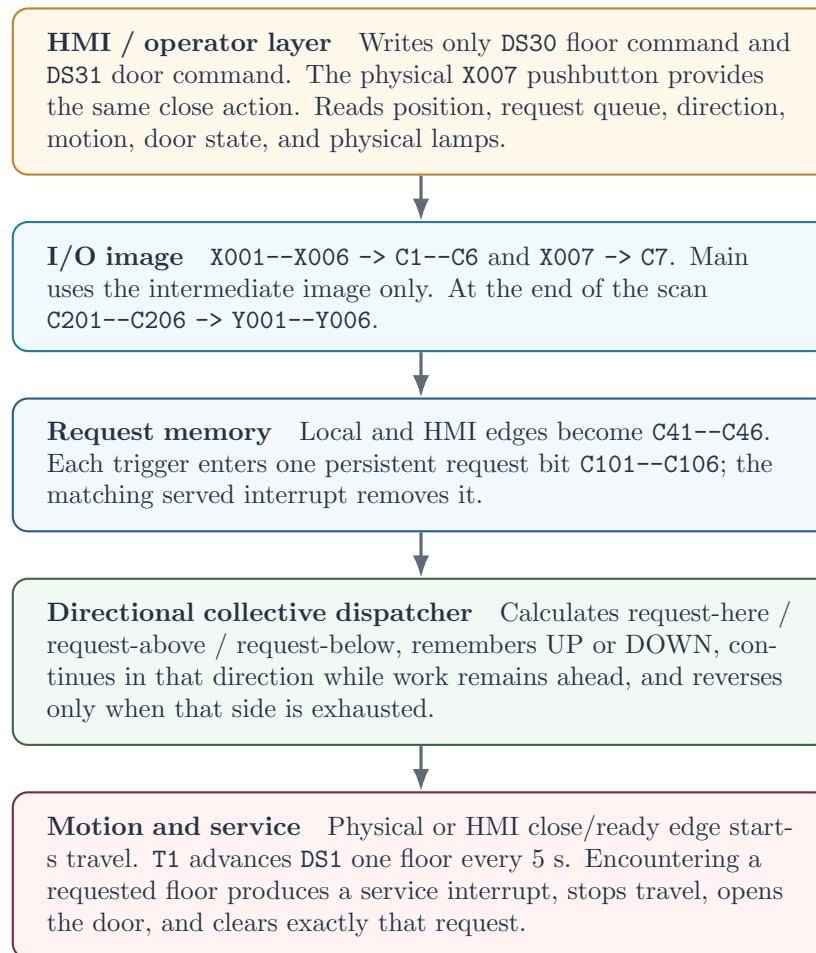
3 Address Map and Signal Ownership

Address	Assigned function	Address	Assigned function
X001--X006	Physical floor buttons 1–6	Y001--Y006	Physical current-floor lamps 1–6
X007	Physical momentary door-close button	C7	Physical door-close input image
C1--C6	Physical floor-button images	C201--C206	Central lamp command bits
C21--C26	Local one-scan floor triggers	C31--C36	HMI one-scan floor triggers
C41--C46	Combined floor triggers	C101--C106	Persistent floor requests
C151--C156	Per-floor served interrupts	C50	HMI close/ready one-scan trigger
C110	Any request pending	C111	Request at current floor
C112	At least one request above	C113	At least one request below
C120	Travel-active hold-in	C121	Start-travel one-scan pulse
C160	Common service interrupt	C161	Stopped/same-floor service pulse
C162	Moving-arrival service pulse	C170	Door-closed hold-in
C171	Moving up status	C172	Moving down status
C174	Door-open status	C401--C414	Reserved ONS storage bits
DS1	Current floor, 1–6	DS3	Direction memory: 0/1/2
DS30	HMI floor command, 0–6	DS31	HMI door command, 0/1
T1	5.0-second inter-floor timer		

Single-writer output rule

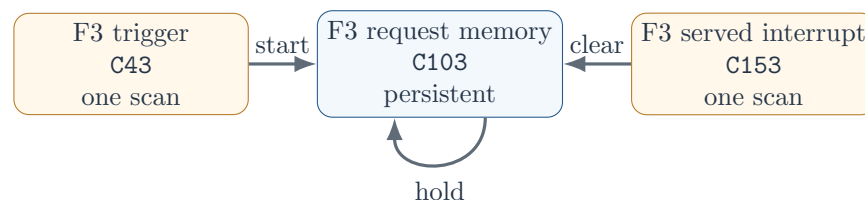
C201--C206 are the only logical commands for the six floor lamps, and each has one **OUT** instruction in Main. Y001--Y006 are written only in the I/O subroutine, one **OUT** instruction per physical output. No request, dispatch, door, or movement rung writes a Y address. For troubleshooting, follow the chain DS1 -> C201--C206 -> Y001--Y006.

4 Program Architecture



5 The Trigger → Memory → Interrupt Pattern

The previous document attempted to draw the request as a branched seal-in rung. That branch drawing was visually ambiguous. This revision teaches the same digital-control idea with a signal-flow diagram and a single Boolean equation. Floor 3 is shown below.



Single-writer request-memory equation

```

(C43 FLOOR3_TRIGGER OR C103 REQUEST_F3) AND /C153 SERVED_F3
--- OUT C103 REQUEST_F3
  
```

The request bit has exactly one OUT coil. A one-scan trigger starts it, its own contact holds it, and the one-scan served interrupt breaks the hold. Repeat the same structure for all six floors.

Why this form is used instead of SET/RESET in this trainer

CLICK SET/RESET instructions can also represent request memory clearly. This document deliberately uses the hold-in equation because it preserves the trainer's "one logical writer per intermediate bit" discipline: each request bit has one output coil in one rung. The conceptual behavior is still trigger, memory, and interrupt.

6 Directional Collective Decision Rules

At any stopped floor, Main derives four summary bits from the request queue:

C110 ANY_REQUEST: at least one of C101--C106 is on.

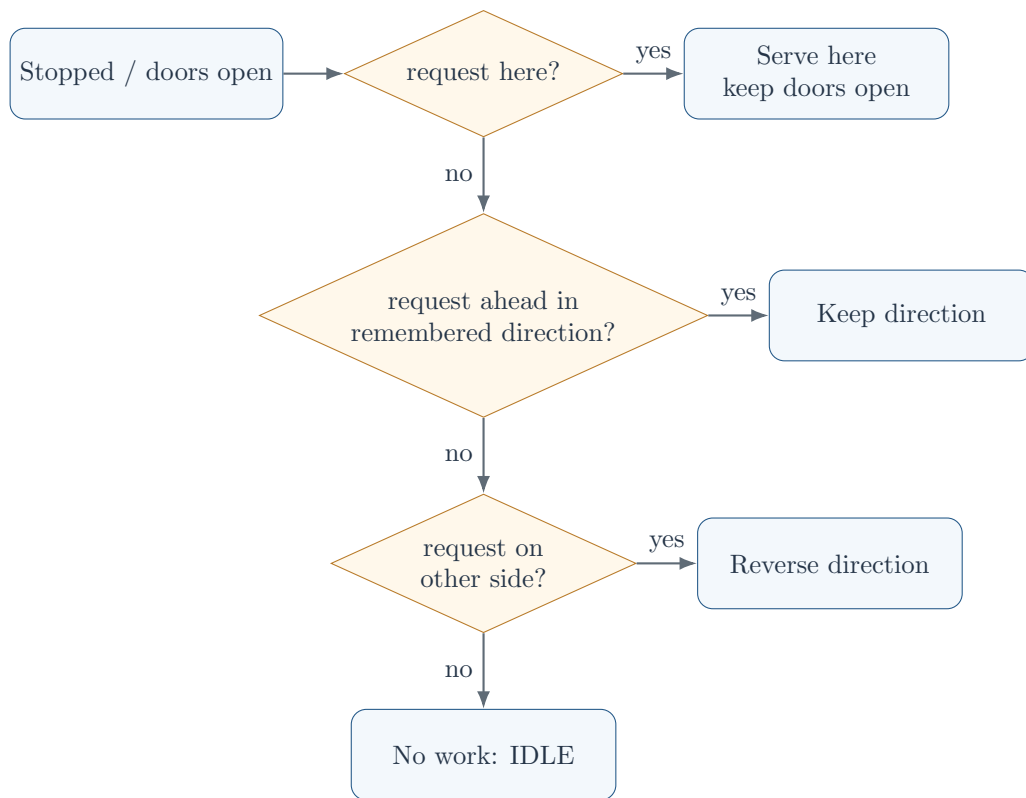
C111 REQUEST_HERE: the request corresponding to DS1 is on.

C112 REQUEST_ABOVE: any requested floor is numerically greater than DS1.

C113 REQUEST_BELOW: any requested floor is numerically less than DS1.

Direction selection is intentionally simple and deterministic:

1. If stopped at a requested floor, serve that floor first; do not depart through an open request at the current floor.
2. If the previous direction is UP and a request remains above, keep UP.
3. If the previous direction is UP, nothing remains above, and something remains below, reverse to DOWN.
4. If the previous direction is DOWN and a request remains below, keep DOWN.
5. If the previous direction is DOWN, nothing remains below, and something remains above, reverse to UP.
6. If idle with requests on only one side, choose that side. If idle with requests on both sides, use UP as the documented tie-breaker.
7. If no request exists anywhere, set direction to IDLE.



7 Main Program - Detailed Rung Plan

Rung order below is chosen so newly created requests are stored before dispatch decisions. The 5-second position step occurs before the moving-arrival comparison, so the service test sees the newly reached floor. A one-scan PLC delay between the service pulse and a hold-in dropping out is insignificant compared with the 5-second travel interval and is called out explicitly during commissioning.

First-scan initialization - place before normal sequencing

Initialize the simulated cabin before any request or dispatch logic is evaluated.

```
SC2 --- COPY 1 to DS1
SC2 --- COPY 0 to DS3
```

Rung 000 - Acquire the physical input image

```
CALL IO
```

Rungs 001–006 - Local button one-scan triggers

Reserve unique ONS storage bits. Do not reuse an ONS storage bit on another edge detector.

```
C1 --- ONS C401 --- OUT C21 LOCAL_F1_TRIGGER
C2 --- ONS C402 --- OUT C22 LOCAL_F2_TRIGGER
C3 --- ONS C403 --- OUT C23 LOCAL_F3_TRIGGER
C4 --- ONS C404 --- OUT C24 LOCAL_F4_TRIGGER
C5 --- ONS C405 --- OUT C25 LOCAL_F5_TRIGGER
C6 --- ONS C406 --- OUT C26 LOCAL_F6_TRIGGER
```

Rungs 007–012 - HMI floor one-scan triggers

```
DS30=1 --- ONS C407 --- OUT C31 HMI_F1_TRIGGER
DS30=2 --- ONS C408 --- OUT C32 HMI_F2_TRIGGER
DS30=3 --- ONS C409 --- OUT C33 HMI_F3_TRIGGER
DS30=4 --- ONS C410 --- OUT C34 HMI_F4_TRIGGER
DS30=5 --- ONS C411 --- OUT C35 HMI_F5_TRIGGER
DS30=6 --- ONS C412 --- OUT C36 HMI_F6_TRIGGER
```

Rungs 013–018 - Combine local and HMI triggers

```
(C21 OR C31) --- OUT C41 FLOOR1_TRIGGER
(C22 OR C32) --- OUT C42 FLOOR2_TRIGGER
(C23 OR C33) --- OUT C43 FLOOR3_TRIGGER
(C24 OR C34) --- OUT C44 FLOOR4_TRIGGER
(C25 OR C35) --- OUT C45 FLOOR5_TRIGGER
(C26 OR C36) --- OUT C46 FLOOR6_TRIGGER
```

Rungs 019–024 - Persistent floor requests

Each request bit is written in exactly one place. Its served interrupt is generated later from the common service pulse and current floor.

```
(C41 OR C101) AND /C151 --- OUT C101 REQUEST_F1
(C42 OR C102) AND /C152 --- OUT C102 REQUEST_F2
(C43 OR C103) AND /C153 --- OUT C103 REQUEST_F3
(C44 OR C104) AND /C154 --- OUT C104 REQUEST_F4
(C45 OR C105) AND /C155 --- OUT C105 REQUEST_F5
(C46 OR C106) AND /C156 --- OUT C106 REQUEST_F6
```

Rung 025 - Physical/HMI door-close / ready trigger

```
(C7 OR DS31=1) --- ONS C413 --- OUT C50 CLOSE_READY_TRIGGER
```

C7 is the isolated image of the X007 normally-open momentary pushbutton. The combined condition must return false before either source can create another rising-edge trigger.

Rungs 026–029 - Request summary bits

These Boolean summaries are the heart of the dispatcher. The explicit comparisons are easy to watch online and require no array or indirect addressing.

```
(C101 OR C102 OR C103 OR C104 OR C105 OR C106) --- OUT C110 ANY_REQUEST
```

```
(DS1=1 AND C101) OR (DS1=2 AND C102) OR (DS1=3 AND C103) OR  
(DS1=4 AND C104) OR (DS1=5 AND C105) OR (DS1=6 AND C106)  
--- OUT C111 REQUEST_HERE
```

```
(DS1<2 AND C102) OR (DS1<3 AND C103) OR (DS1<4 AND C104) OR  
(DS1<5 AND C105) OR (DS1<6 AND C106)  
--- OUT C112 REQUEST_ABOVE
```

```
(DS1>1 AND C101) OR (DS1>2 AND C102) OR (DS1>3 AND C103) OR  
(DS1>4 AND C104) OR (DS1>5 AND C105)  
--- OUT C113 REQUEST_BELOW
```

Rungs 030–036 - Direction memory at a close/ready event

The current direction is retained whenever useful. Only the conditions requiring a change execute a COPY. A newly idle car chooses the only populated side; if both sides are populated, UP is the classroom tie-breaker.

```
/C120 /C110 --- COPY 0 to DS3 ; IDLE
```

```
C50 DS3=1 /C112 C113 /C111 --- COPY 2 to DS3 ; UP exhausted -> DOWN  
C50 DS3=2 /C113 C112 /C111 --- COPY 1 to DS3 ; DOWN exhausted -> UP
```

```
C50 DS3=0 C112 /C113 /C111 --- COPY 1 to DS3 ; only above  
C50 DS3=0 C113 /C112 /C111 --- COPY 2 to DS3 ; only below  
C50 DS3=0 C112 C113 /C111 --- COPY 1 to DS3 ; both sides: tie -> UP
```

If DS3=1 and C112=1, or DS3=2 and C113=1, no direction write is needed: the remembered direction is already correct.

Rung 037 - Start-travel trigger

A close/ready edge may start movement only if some request exists away from the current floor and a valid direction has been established.

```
C50 C110 /C111 (DS3=1 OR DS3=2) --- OUT C121 START_TRAVEL_PULSE
```

Rungs 038–039 - Door and travel hold-ins

Both states use the same trigger/hold/interrupt teaching pattern. The service interrupt is computed later in the scan; therefore the hold-in normally drops on the following scan, only a few milliseconds later.

```
((C50 AND /C111) OR C170) AND /C160 --- OUT C170 DOOR_CLOSED  
(C121 OR C120) AND /C160 --- OUT C120 TRAVEL_ACTIVE
```


Rung 040 - Five-second inter-floor pace

```
C120 /T1 --- TMR T1, 5.0 second, ON-delay, automatic reset
```

Rungs 041–042 - Move exactly one floor per timer pulse

```
T1 DS3=1 DS1<6 --- MATH DS1 = DS1 + 1
T1 DS3=2 DS1>1 --- MATH DS1 = DS1 - 1
```

Rungs 043–045 - Generate service pulses

The moving service condition is evaluated *after* the floor step, so a request is detected at the floor just reached. The stopped condition handles a request for the floor where the car is already parked.

```
/C120 C111 --- ONS C414 --- OUT C161 IDLE_SAME_FLOOR_SERVICE

C120 T1 ((DS1=1 AND C101) OR (DS1=2 AND C102) OR (DS1=3 AND C103) OR
        (DS1=4 AND C104) OR (DS1=5 AND C105) OR (DS1=6 AND C106))
        --- OUT C162 MOVING_ARRIVAL_SERVICE

(C161 OR C162) --- OUT C160 SERVICE_INTERRUPT
```

Rungs 046–051 - Decode the served floor

These pulses clear exactly one request on the next scan through the request hold-in rungs.

```
C160 DS1=1 --- OUT C151 SERVED_F1
C160 DS1=2 --- OUT C152 SERVED_F2
C160 DS1=3 --- OUT C153 SERVED_F3
C160 DS1=4 --- OUT C154 SERVED_F4
C160 DS1=5 --- OUT C155 SERVED_F5
C160 DS1=6 --- OUT C156 SERVED_F6
```

Rungs 052–054 - Direction and door status

```
C120 DS3=1 --- OUT C171 MOVING_UP
C120 DS3=2 --- OUT C172 MOVING_DOWN
/C170 --- OUT C174 DOOR_OPEN
```

Rungs 055–060 - Centralized floor-lamp command decode

The floor lamps represent the car's current simulated position. Requests are shown separately on the HMI from C101--C106.

```
DS1=1 --- OUT C201 FLOOR1_LAMP_CMD
DS1=2 --- OUT C202 FLOOR2_LAMP_CMD
DS1=3 --- OUT C203 FLOOR3_LAMP_CMD
DS1=4 --- OUT C204 FLOOR4_LAMP_CMD
DS1=5 --- OUT C205 FLOOR5_LAMP_CMD
DS1=6 --- OUT C206 FLOOR6_LAMP_CMD
```

Rungs 061–062 - End-of-scan I/O and END

The final I/O call applies the newly calculated lamp image in the same scan.

```
CALL IO
END
```

8 I/O Subroutine - Only Physical I/O Page

IO Rungs 000–006 - Physical input image

```
X001 --- OUT C1 FLOOR1_BUTTON_IMAGE
X002 --- OUT C2 FLOOR2_BUTTON_IMAGE
X003 --- OUT C3 FLOOR3_BUTTON_IMAGE
X004 --- OUT C4 FLOOR4_BUTTON_IMAGE
X005 --- OUT C5 FLOOR5_BUTTON_IMAGE
X006 --- OUT C6 FLOOR6_BUTTON_IMAGE
X007 --- OUT C7 DOOR_CLOSE_BUTTON_IMAGE
```

IO Rungs 007–012 - Physical floor lamps

Each physical output appears exactly once in the entire project.

```
C201 --- OUT Y001 FLOOR1_LAMP
C202 --- OUT Y002 FLOOR2_LAMP
C203 --- OUT Y003 FLOOR3_LAMP
C204 --- OUT Y004 FLOOR4_LAMP
C205 --- OUT Y005 FLOOR5_LAMP
C206 --- OUT Y006 FLOOR6_LAMP
```

IO Rung 012 - Return

```
RETURN
```

Why I/O is called at both ends of Main

The first call acquires the latest button states into C1--C6. Main then performs all logic with intermediate variables. The final call transfers the newly decoded C201--C206 image to Y001--Y006 in the same scan. This keeps hardware ownership centralized without adding a visible one-scan lamp delay.

9 How a Complete Up/Down Sweep Works

Assume the car is at Floor 1 with doors open. Requests F3 and F6 are entered, then the HMI produces a close/ready edge. C112=1, so direction becomes UP and travel starts. Five seconds later the car reaches Floor 2. If Floor 2 is not requested, it does not stop. At 10 seconds it reaches Floor 3; because C103=1, C162 creates a service interrupt. Travel stops, the door opens, and the F3 request clears. After the operator/HMI makes the next close/ready edge, C112 is still true because F6 remains above, so the car keeps UP. It passes unrequested Floors 4 and 5 and serves Floor 6.

If F2 was requested while the car was already above Floor 2, C102 remains pending. At Floor 6 there is no request above but there is one below, so on the next close/ready event the dispatcher changes DS3 from UP to DOWN. The car then sweeps downward and serves F2 without an unnecessary intermediate reversal.

Time	Current floor	Direction	Event	Pending example
0 s	1	UP	depart after close/ready	F3, F6
5 s	2	UP	pass if not requested	F3, F6
10 s	3	UP	serve / door open	F6
15 s after next close	4	UP	pass	F6
20 s	5	UP	pass	F6
25 s	6	UP	serve / door open	maybe F2 below
next departure	6	DOWN	reverse because no requests above	F2

10 Important Edge Cases

10.1 New request appears ahead while moving

It is latched immediately. When the car reaches that floor, the moving-arrival service rung sees the request and stops there. No new target-selection event is required.

10.2 New request appears behind while moving

It remains latched but does not cause an immediate reversal. The car finishes the current-direction sweep, then reverses when no request remains ahead. This is the behavior that prevents “ping-pong” motion.

10.3 Request for the current floor

When the car is stopped, C111 becomes true. C161 produces a one-scan service pulse; the corresponding request clears and the car does not move. If the HMI attempts to close at the same time, the start-travel rung is blocked by C111.

10.4 Request arrives for a floor just passed

The request stays pending for the return sweep. The program never decrements position simply because a late request appeared behind the car.

10.5 Queue becomes empty

Once the last request is serviced and C110 becomes false, the stopped-idle rung copies 0 to DS3. The car remains parked at its last served floor with the simulated door open.

11 Recommended FUXA Screen

Use one vertical six-floor graphic, six floor-request buttons, six pending-request indicators, and a small status panel. The pending indicators are important because they make the collective-control queue visible to students.

FUXA tag	CLICK item	Use
HMI_FloorCommand	DS30	Writable 0–6 momentary floor request
HMI_DoorCommand	DS31	Writable 0=open/not ready, 1=close/ready
Physical_DoorClose	X007	Normally-open momentary pushbutton; same action as HMI close
CurrentFloor	DS1	Read-only floor number
Direction	DS3	Read-only 0 idle, 1 up, 2 down
TravelActive	C120	Read-only cabin movement animation
DoorClosed	C170	Read-only simulated closed-door state
DoorOpen	C174	Read-only simulated open-door state
Request1--Request6	C101--C106	Read-only pending queue indicators
MovingUp / MovingDown	C171 / C172	Read-only directional arrows
FloorLamp1--FloorLamp6	Y001--Y006	Read-only actual physical lamp image

Modbus addressing

Use the CLICK Address Picker in the installed CLICK Programming Software to confirm exact Modbus coil/register offsets for the CPU and software version being used. The HMI should write only its command variables DS30 and DS31; PLC-owned state and output variables should remain read-only from FUXA.

12 Verification Tests for Directional Collective Behavior

Test action	Expected result
PLC enters RUN	DS1=1, DS3=0, door open, and only Y001 is on.
Tap X004	Local trigger pulses once; combined F4 trigger pulses once; C104 remains on after release.
HMI pulses DS30=6	HMI F6 trigger pulses once and C106 stays pending after DS30 returns to 0.
Queue F3, F6, F4, F2 at Floor 1	All four request bits remain stored simultaneously.
Issue close/ready at Floor 1	Direction becomes UP and travel starts; the car does not choose a single fixed final target.
Tap X007 after a door-open stop	C7 follows the press, C50 pulses for one scan, and travel starts when an away-floor request is pending.
Sweep with F2, F3, F4, F6 pending	Car services 2, 3, 4, then 6 in ascending order, with a separate close/ready event after each door-open stop.
Add F5 while car is moving from F3 toward F6	F5 request latches and is serviced on the same upward sweep.
Add F2 after the car has passed F2 upward	F2 remains pending; car does not reverse early; F2 is served after upper requests are exhausted and direction changes to DOWN.
Request current floor while stopped	Same-floor service pulse clears that request without any 5-second movement.
No requests remain after a stop	C110=0; direction returns to DS3=0; car remains parked.
Hold a physical button	Only the initial ONS edge triggers. Release and press again to create a new request after service.
Inspect output writers	Each C201--C206 has one OUT coil in Main and each Y001--Y006 has one OUT coil in IO.

13 Commissioning Watch List

A useful online watch list is C1--C7, X007, C21--C26, C31--C36, C41--C46, C101--C106, C110--C113, C151--C156, C50, C120--C121, C160--C162, C170--C174, DS1, DS3, DS30--DS31, T1, C201--C206, and Y001--Y006.

Troubleshoot in this order:

1. Verify the floor buttons reach C1--C6 and the physical door-close button X007 reaches C7.
2. Verify the ONS produces exactly one local or HMI trigger.
3. Verify the combined trigger sets the matching persistent request through its one-writer hold-in rung.
4. Verify C111, C112, and C113 correctly describe requests relative to DS1.
5. Verify an X007/C7 or HMI close/ready edge creates C50, establishes the intended DS3, and creates C121 only when the current floor is not itself pending.
6. Verify C120 holds travel active and T1 causes exactly one floor step every 5.0 seconds.
7. Verify a requested floor reached while moving creates C162, then C160, then exactly one C15x served interrupt.
8. Verify the matching request clears and the door-open status appears at the service stop.
9. Verify direction is retained when work remains ahead and changes only after that side of the queue is exhausted.
10. Finally verify DS1 -> C201--C206 -> Y001--Y006 for the floor lamps.

CLICK implementation check

The rung text is an implementation plan, not an imported CLICK project file. Enter the logic with the actual CLICK compare, ONS, OUT, TMR, MATH, COPY, CALL, and RETURN instructions supported by the selected CPU. Confirm the timer time base, first-scan system bit, ONS behavior, and Modbus addresses in the installed CLICK software. Every ONS storage bit must be unique.

14 Teaching Summary

The central lesson is not merely “move to a selected floor.” It is how a PLC turns momentary events into persistent work, then schedules that work without needless reversals:

button/HMI edge → request memory → directional sweep → service interrupt → request cleared

The same discipline is visible at every level: trigger signals are short, requests are persistent, the current direction is remembered, physical I/O is isolated through intermediate variables, and each physical output has one obvious writer. This gives students a realistic scheduling problem while preserving a program structure that is easy to observe and debug online.

Natural next expansion

After this six-destination trainer is working, the next realistic extension is to split hall calls into separate UP and DOWN request sets. Then an upward-moving car can collect UP hall calls and cabin destinations while deferring opposite-direction hall calls until the return sweep. The request-memory and centralized-I/O architecture in this document can remain intact.